

Frontend que não vira bagunça

Encontro 1 — Estado, renderização e a camada de comunicação

2h

de aula

24min

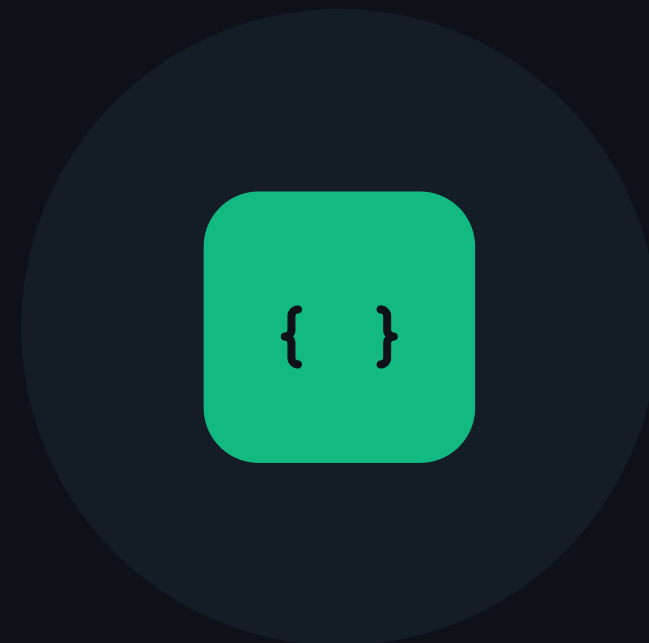
de exposição

96min

de código

3

práticas



Como as próximas duas horas funcionam

Regra da casa: eu falo pouco, vocês programam muito. Toda exposição cabe em 8 minutos.

MIN 00–10

Aquecimento + demo

Cinco perguntas rápidas e a demonstração do que existirá ao fim do dia.

MIN 10–48

Bloco 1 — Estado

Micro-aula, Prática 1 (completar o render) e votação em dupla.

MIN 48–84

Bloco 2 — Derivado

Micro-aula, Prática 2 (busca reativa).

MIN 84–113

Bloco 3 — CSS

Micro-aula mobile-first e Prática 3 (cartão responsivo).

Nas práticas: programação em par. Um digita (driver), o outro lê e questiona (navigator). Troca a cada 12 minutos — o timer fica projetado. Quem só assiste não aprende.

O que existirá no seu computador na sexta-feira

HOJE

Uma tela que reage

- Lista de pacientes vinda de um arquivo JSON
- Busca que filtra enquanto você digita
- Cartão responsivo: 1 coluna no celular, 3 no monitor
- Zero framework de JavaScript

ENCONTRO 2

Uma API de verdade

- Servidor Node/Express em TypeScript
- Rotas GET e POST devolvendo JSON
- Banco SQLite com SQL escrito por você
- O frontend trocando o arquivo falso pela API — em UMA linha

Mini-Prontuário: pacientes e atendimentos. O vocabulário (Patient, Encounter) vem do HL7 FHIR, o padrão internacional de interoperabilidade em saúde. Não vamos implementar FHIR — vamos nomear como o mercado nomeia.

Três regras que valem para você e para a IA

Escopo fechado de bibliotecas

express, better-sqlite3, tsx, typescript e Bootstrap por CDN. Mais nada. Restringir escopo é competência profissional, não limitação — todo time de verdade tem uma lista de dependências aprovadas.

IA é permitida e incentivada

Modo conversa (ChatGPT, Claude, Gemini). Ainda não usamos agentes autônomos: sem testes e sem arquitetura, não há rede de segurança para segurar o que eles produzem.

Você precisa saber explicar

Qualquer linha do seu projeto pode ser apontada e perguntada. Código que você não sabe explicar é código que você não tem.

Sem consultar nada. Sem nota. Só para o cérebro acordar.

- 1 O que `document.querySelector('#lista')` devolve?
- 2 O que `addEventListener('input', fn)` faz — e quando `fn` roda?
- 3 O que `[1,2,3].map(x => x * 2)` devolve? E o array original, muda?
- 4 `fetch()` devolve o dado pronto ou uma promessa?
- 5 O que é um breakpoint em CSS?

Tentar lembrar antes de reler é a técnica de estudo com maior evidência de eficácia. Errar aqui é parte do método.

BLOCO 01

Por que sua tela mente

O bug mais caro do frontend não é de sintaxe. É de arquitetura.

8 min · exposição



O contador diz 6. A lista mostra 5.

Você inativou um paciente. O cartão sumiu da lista. O contador continuou igual. Nada quebrou, nada deu erro no console — e o sistema está mentindo.

O QUE O PROGRAMADOR ESCREVEU

```
// no clique do botao Inativar
botao.addEventListener('click', () => {
  cartao.remove();           // mexe na tela
  contador.textContent = total - 1; // mexe na tela
});

// em outro arquivo, na busca
campo.addEventListener('input', () => {
  // redesenha a lista a partir dos dados
  desenharLista(pacientes); // o inativo VOLTA
});
```

DIAGNÓSTICO

Existem duas verdades

Uma no array `pacientes`, que ninguém atualizou. Outra no DOM, que foi alterado à mão.

Enquanto ninguém redesenha, as duas convivem. No instante em que algo redesenha, a verdade esquecida volta e apaga a outra.

Esse bug não aparece em teste rápido. Aparece em produção, na frente do cliente, e é irreproduzível — porque depende da ORDEM em que o usuário clicou nas coisas.

O quadro branco da recepção

O JEITO ERRADO

Só existe o quadro branco

A recepcionista apaga e escreve direto no quadro conforme os pacientes chegam e saem.

Quando alguém pergunta “quantos estão esperando?”, a resposta é: conta aí no quadro.

Se duas pessoas escrevem no quadro em momentos diferentes, ninguém sabe mais qual versão é a certa. E se o quadro cair, acabou.

O JEITO CERTO

Existe um livro de registro

Tudo é anotado no livro. O quadro branco é reescrito a partir do livro, sempre inteiro.

Quando alguém pergunta “quantos estão esperando?”, a resposta vem do livro — nunca do quadro.

O quadro pode cair, ser apagado, mudar de lugar. O livro continua sendo a verdade.

O livro de registro é o ESTADO. O quadro branco é o DOM. O DOM é descartável — o estado, não.

A REGRA DA SEMANA

**O estado muda.
A tela é consequência.**

Nunca o contrário. A tela nunca é a fonte da verdade.

Um sentido só, sempre o mesmo caminho



O QUE ISSO TE DÁ

Um lugar só para procurar

Tela errada? Ou o estado está errado, ou o render está errado. Duas hipóteses, não vinte. O bug para de ser caçada e vira dedução.

O QUE ISSO TE CUSTA

Mais código no começo

Escrever três arquivos para mostrar oito nomes parece exagero. É. E deixa de ser no dia em que a tela tiver quatro filtros e um formulário.

BLOCO 02

Anatomia do projeto

Quatro arquivos. Cada um com uma frase que define o que ele pode fazer.



Quatro arquivos, quatro responsabilidades

Se você souber responder “de quem é essa responsabilidade?”, 80% das dúvidas desta semana já estão resolvidas.

APP.JS

O maestro

Escuta eventos, chama ações, manda redesenhar. Não guarda dado, não desenha nada sozinho.

STATE.JS

O livro de registro

Guarda o estado e as ações. NÃO conhece o DOM — se aparecer ``document`` aqui, algo saiu do lugar.

RENDER.JS

O desenhista

Recebe o que deve aparecer e coloca na tela. NÃO decide nada: não filtra, não ordena, não calcula regra.

API.JS

A porta única

O único arquivo autorizado a chamar ``fetch``. Ninguém mais no frontend sabe que existe HTTP.

Isto não é “arquitetura”. É só arrumação. Arquitetura de verdade começa na Semana 3 — e vai fazer muito mais sentido depois de você ter sentido falta dela.

O estado: uma resposta para “o que a tela precisa mostrar agora?”

PUBLIC/JS/STATE.JS

```
const state = {  
  patients: [],           // veio do servidor  
  searchTerm: '',         // o que o usuario digitou  
  onlyActive: false,     // o filtro esta ligado?  
  isLoading: true,       // ainda estamos buscando?  
  errorMessage: null,    // deu ruim? o que dizer?  
};  
  
// A UNICA funcao autorizada a mudar `patients`  
export function setPatients(patients) {  
  state.patients = patients;  
  state.isLoading = false;  
  state.errorMessage = null;  
  notify();           // avisa quem quiser saber  
}
```

TESTE MENTAL

Apague o index.html

Este arquivo ainda faria sentido? Tem que fazer. Ele descreve o problema, não a página.

POR QUE `NOTIFY()`

Ninguém precisa se lembrar

Sem isso, toda ação teria que lembrar de mandar redesenhar. Uma esquecida e a tela dessincroniza — exatamente o bug do começo da aula.

Repare no que NÃO está guardado

A lista filtrada não está no estado. E isso é uma decisão, não um esquecimento.

FATO

Precisa ser guardado

- A lista completa de pacientes
- O texto digitado na busca
- Se o filtro de ativos está ligado

CONSEQUÊNCIA

É calculado na hora

- A lista que aparece na tela
- Quantos resultados apareceram
- Se deve mostrar “nada encontrado”

Guardar consequência é criar uma segunda verdade — e as duas vão discordar no dia em que você esquecer de atualizar uma delas.
A pergunta que resolve: isto é um FATO, ou é o RESULTADO de outros fatos?

O desenhista é burro de propósito

PUBLIC/JS/RENDER.JS

```
function patientCardTemplate(patient) {  
  const badge = patient.active  
    ? 'status-badge--active'  
    : 'status-badge--inactive';  
  
  return `  
    <li class="patient-card">  
      <h2>${escapeHtml(patient.name)}</h2>  
      <span class="status-badge ${badge}">...</span>  
    </li>`;  
}  
  
export function renderPatientList(patients, term, box) {  
  if (patients.length === 0) {  
    box.innerHTML = emptyStateTemplate(term);  
    return;  
  }  
  box.innerHTML = patients.map(patientCardTemplate).join('');
```

NOTE O QUE FALTA

Nenhum `if` de negócio

Ele não decide quem aparece. Recebe a lista pronta e desenha. Quem decidiu foi o estado.

ESCAPEHTML

Por que escapar o texto?

Vamos montar HTML com `innerHTML`. Se um nome fosse `

Isso se chama XSS. Voltamos a ele em OWASP Top 10.

Redesenhar a lista INTEIRA a cada tecla digitada? Sério?

SIM, SÉRIO

Para 8 pacientes

É instantâneo. Você não consegue perceber. E o código fica trivial: uma linha que joga fora tudo e desenha de novo.

Não existe estado intermediário para dessincronizar.

ONDE DÓI

Para 10 mil linhas

Aí o navegador engasga. E pior: o cursor sai do campo de texto, a rolagem volta ao topo, o vídeo que estava tocando reinicia.

Redesenhar tudo destrói o que o usuário estava fazendo.

A SAÍDA

É aqui que o React nasce

O virtual DOM existe para dizer “redesenehe tudo” e, por baixo, mexer só no que mudou.

Você vai entender React muito melhor por ter vivido esta versão primeiro.

Aprender a ferramenta antes do problema é decorar. Sentir o problema e depois receber a ferramenta é entender.

O maestro: liga evento a ação, e estado a tela

PUBLIC/JS/APP.JS

```
// 1. eventos do usuario viram ACOES
searchInput.addEventListener('input', (event) => {
  setSearchTerm(event.target.value); // nao mexe no DOM!
});

// 2. a UNICA funcao que desenha a tela inteira
function renderApp(state) {
  if (state.errorMessage) { renderError(...); return; }
  if (state.isLoading) { renderLoading(...); return; }

  renderPatientList(state.visiblePatients, ...);
  renderCounter(...);
}

// 3. sempre que o estado mudar, redesenha
subscribe(renderApp);
```

REPARE NA LINHA 3

O listener não desenha

Ele só avisa o estado. Quem desenha é o `subscribe`, depois. Sempre nessa ordem.

ORDEM IMPORTA

Erro → carregando → dados

Estados de exceção primeiro, com `return`. Assim a última linha só roda quando tudo está bem — e fica limpa.

A porta única para o mundo lá fora

PUBLIC/JS/API.JS

```
// ENCONTRO 1: um arquivo estatico
const PATIENTS_URL = './mock/patients.json';

export async function listPatients() {
  const response = await fetch(PATIENTS_URL);

  // fetch NAO lanca erro em 404 nem em 500.
  // Um 404 chega aqui como resposta bem-sucedida.
  if (!response.ok) {
    throw new Error(`Falha (HTTP ${response.status})`);
  }

  return response.json();
}
```

A PROMESSA

Uma linha vai mudar

No Encontro 2, `PATIENTS\_URL` vira `/api/patients`.

Mais nada no frontend muda. Guardem essa promessa — vou cobrar.

A PERGUNTA

Quantos lugares?

Se a URL mudasse, quantos arquivos você abriria? Um. É essa a razão de existir deste arquivo.

`fetch` não lança erro quando o servidor recusa

Esta é, com folga, a pegadinha que mais derruba gente em entrevista técnica de frontend.

O QUE QUASE TODO MUNDO ESCRIVE

```
try {  
  const r = await fetch('/api/patients');  
  const dados = await r.json();  
  mostrar(dados);  
} catch (erro) {  
  // NUNCA cai aqui em um 404.  
  mostrarErro(erro);  
}
```

O QUE FUNCIONA

```
const r = await fetch('/api/patients');  
  
if (!r.ok) {  
  // 200-299 => ok = true  
  throw new Error(`HTTP ${r.status}`);  
}  
  
const dados = await r.json();  
mostrar(dados);
```

A REGRA EXATA

`fetch` só rejeita quando a mensagem não chega

Sem internet, DNS quebrado, servidor fora do ar, bloqueio de CORS — aí sim ele lança.

Mas 404 (“não achei”), 400 (“seu pedido está errado”) e 500 (“eu quebrei”) são respostas que CHEGARAM. Para o `fetch`, a conversa foi um sucesso: ele perguntou e recebeu resposta. Cabe a você julgar o conteúdo dela.

Fazer a lista aparecer

Troca de driver em 12 min. O código já está no seu computador — vocês vão preencher duas lacunas.

1 Leia antes de escrever

Abra `state.js` e `render.js` inteiros. Não escreva nada nos primeiros 4 minutos.

2 TODO STATE-1

Em `state.js`: guarde a lista, encerre o loading, limpe o erro, chame `notify()`.

3 TODO RENDER-1

Em `render.js`: lista vazia → empty state; senão, map + join("") no innerHTML.

4 Confira no navegador

8 cartões, o contador preenchido, o badge “Inativo” em dois deles.

SE TRAVAR

Perguntas que destravam

- O console mostra algum erro em vermelho?
- ``setPatients`` está sendo chamado? (`console.log`)
- ``notify()`` foi esquecido no fim da função?
- O container é o `<ul id="patient-list">``?

TERMINOU ANTES?

Nível 3

- Faça a lista aparecer ordenada por nome
- Mostre a idade calculada a partir de `birthDate`
- Nenhum dos dois no estado: ambos derivados

O contador de ativos aparece em dois lugares da tela. Ao inativar um paciente, um atualiza e o outro não. A causa mais provável é:

- A** O CSS está sobrescrevendo o valor exibido.
- B** Existem duas fontes da verdade: o estado e o próprio DOM.
- C** O `addEventListener` foi registrado duas vezes.
- D** O navegador está servindo uma versão em cache.

Vote → discuta com a dupla por 2 minutos → vote de novo. Convencer alguém é o teste real de que você entendeu.

BLOCO 03

Busca que reage

O primeiro pedaço de programação reativa da sua vida.



“É só esconder quem não bate com a busca”

PARECE RESOLVER

```
campo.addEventListener('input', (e) => {  
  const termo = e.target.value.toLowerCase();  
  
  document.querySelectorAll('.patient-card')  
    .forEach((cartao) => {  
    const nome = cartao.textContent.toLowerCase();  
    cartao.style.display =  
      nome.includes(termo) ? 'block' : 'none';  
  });  
});
```

O QUE QUEBRA

Três problemas de uma vez

- O contador não sabe de nada — ele conta o estado, e o estado não mudou.
- Chegou um paciente novo? Ele nasce visível, ignorando o filtro.
- Você está lendo dado de volta do DOM (`textContent`): a tela virou banco de dados.

Funciona na demonstração. Falha na segunda funcionalidade. É assim que nasce o código que “ninguém entende como funciona”.

O filtro é um fato novo no estado — a tela só obedece

STATE.JS

```
// 1. o fato novo entra no estado
export function setSearchTerm(term) {
  state.searchTerm = term;
  notify();
}

// 2. a lista visível é CALCULADA, nunca guardada
export function getVisiblePatients() {
  const term = state.searchTerm.trim().toLowerCase();

  return state.patients.filter((patient) => {
    const matchesTerm =
      patient.name.toLowerCase().includes(term);
    const matchesStatus =
      state.onlyActive ? patient.active : true;

    return matchesTerm && matchesStatus;
  });
}
```

FUNÇÃO PURA

Mesma entrada, mesma saída

Não altera nada, não depende de relógio nem de rede. Fácil de testar e fácil de confiar.

DE GRAÇA

Os filtros se combinam sozinhos

Busca e “só ativos” não precisam se conhecer. Cada um é uma condição no mesmo `filter`.

Adicionar um terceiro filtro é acrescentar uma linha.

Busca reativa

Trocaram de driver? Quem digitou na Prática 1 agora só fala.

1 TODO STATE-2

Implemente ``setSearchTerm(term)``. Duas linhas.

2 Ligue o campo

Em `app.js` o listener já existe. Confira que ele chama a ação, e não o DOM.

3 Teste o vazio

Digite “zzz”. Tem que aparecer o empty state, e o contador tem que dizer 0 de 8.

4 Ligue o segundo filtro

Marque “apenas ativos” e busque ao mesmo tempo. Os dois devem se combinar.

CRITÉRIO DE PRONTO

Como saber que acabou

- Digitar filtra sem apertar nada
- O contador acompanha o filtro
- Apagar a busca traz todo mundo de volta
- Nenhum ``style.display`` no seu código

NÍVEL 3

Para quem terminou

- Busca também pelo CNS, não só pelo nome
- Ignore acentos na comparação (normalize)
- Mostre o termo buscado dentro do empty state

BLOCO 04

CSS que não vira lixo

Mobile-first, tokens, BEM e uma biblioteca padrão de indústria.



A analogia da mala de mão

DESKTOP-FIRST

Arrumar a mala grande e ir tirando

Você arruma tudo o que quer levar. Depois descobre que só cabe a mala de mão e começa a tirar coisas.

O resultado é uma mala cheia de decisões desfeitas. Em CSS: dezenas de ``max-width`` desligando regras que você mesmo acabou de escrever.

MOBILE-FIRST

Arrumar a mala pequena e ir somando

Você começa pelo essencial: o que não pode faltar de jeito nenhum. Se sobrar espaço, acrescenta.

Em CSS: o estilo base é o do celular, e cada ``min-width`` ACRESCENTA. Nenhuma regra é desfeita — e a versão mais restrita é a que recebeu mais atenção.

No Brasil, a maior parte do acesso à web é por celular. Mobile-first não é preferência estética: é onde seu usuário está.

A mesma tela, escrita dos dois jeitos

DESKTOP-FIRST (EVITE)

```
.patient-list {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}  
  
@media (max-width: 1199px) { /* desfaz */  
  .patient-list {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (max-width: 767px) { /* desfaz de novo */  
  .patient-list {  
    grid-template-columns: 1fr;  
  }  
}
```

MOBILE-FIRST (NOSSO PADRÃO)

```
.patient-list {  
  display: grid;  
  grid-template-columns: 1fr; /* o essencial */  
}  
  
@media (min-width: 768px) { /* acrescenta */  
  .patient-list {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (min-width: 1200px) { /* acrescenta */  
  .patient-list {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

O TESTE DE 5 SEGUNDOS

Abra seu CSS e procure por ``max-width``

Se houver muitos, você desenhrou para o monitor e está remendando para o celular. Inverter a ordem das media queries costuma reduzir o arquivo pela metade.

Uma decisão de design com nome

Se você digitou #12b981 no meio de um componente, criou um valor mágico. Se digitou var(--cor-marca), criou vocabulário.

PUBLIC/CSS/TOKENS.CSS

```
:root {  
  /* superficies */  
  --cor-fundo: #f6f7f9;  
  --cor-superficie: #ffffff;  
  
  /* semantica: o nome diz o PAPEL, nao a cor */  
  --cor-marca: #12b981;  
  --cor-alerta: #ff6b4a;  
  
  /* escala de espacamento: multiplos de 4px */  
  --espaco-2: 0.5rem; /* 8px */  
  --espaco-4: 1rem; /* 16px */  
  --espaco-5: 1.5rem; /* 24px */  
  
  --raio-md: 12px;  
}
```

POR QUE NOME SEMÂNTICO

`--cor-marca`, não `--verde`

No dia em que a identidade virar azul, você troca uma linha. Se o token se chamasse `--verde`, você teria `--verde: azul`.

POR QUE ESCALA FIXA

Nunca invente um valor

13px, 15px, 17px espalhados pela folha são o motivo de o layout “parecer torto sem saber por quê”.

Múltiplos de 4 dão ritmo visual de graça.

Um nome que responde a três perguntas sozinho

PUBLIC/CSS/COMPONENTS.CSS

```
/* BLOCO: o componente inteiro */  
.patient-card { ... }  
  
/* ELEMENTO: parte que so existe dentro do bloco */  
.patient-card__name { ... }  
.patient-card__meta { ... }  
  
/* MODIFICADOR: uma variacao do bloco */  
.patient-card--inactive { ... }
```

DE QUEM É?

`patient-card` — do cartão de paciente. Não é de qualquer cartão.

O QUE É?

`\_\_name` — a parte do nome. Sozinho não faz sentido.

EM QUE ESTADO?

`--inactive` — continua sendo um cartão, só que inativo.

O que BEM evita: `.card .titulo span.ativo > p`. Esse seletor obriga você a manter uma estrutura de HTML na cabeça, e quebra no dia em que alguém mexer no markup.

Bootstrap 5.3 — cinco ideias que explicam tudo

Padrão de indústria há mais de uma década. Você vai encontrá-lo em sistema interno, em painel administrativo e em muita vaga de emprego.

1

Mobile-first por construção

Toda media query do Bootstrap é ``min-width``. Os prefixos ``sm md lg xl`` significam “a partir de”, nunca “até”.

2

Grid de 12 colunas

``container`` → ``row`` → ``col-md-6``. Você compõe o layout em vez de posicionar elemento por elemento.

3

Componentes com nome

``card``, ``navbar``, ``badge``, ``alert``, ``form-control``. Um vocabulário que designer e desenvolvedor compartilham.

4

Utilitários atômicos

``d-flex``, ``gap-3``, ``mt-4``, ``text-muted``. Ajuste fino sem abrir o arquivo CSS.

5

Tematizável por variável CSS

Desde a 5.3 dá para trocar ``--bs-primary`` direto, sem recompilar Sass.

O que é do Bootstrap e o que é seu

BOOTSTRAP RESOLVE

O esqueleto e o mobiliário genérico

- Grid, container, breakpoints
- Botões, campos de formulário, badges
- Utilitários de espaçamento e flex
- Reset e tipografia base

VOCÊ ESCRIVE

O componente do seu domínio

- `.patient-card`` — só existe neste sistema
- `.status-badge--inactive``
- Os tokens da identidade visual
- As regras de layout específicas da tela

Se você usar **SÓ** Bootstrap, seu sistema fica com cara de template. Se você não usar **NADA**, gasta duas semanas reescrevendo grid. O trabalho profissional é saber onde fica a fronteira.

Quando NÃO usar Bootstrap — e por que não escolhemos Tailwind

EVITE BOOTSTRAP

Três situações

- Produto que precisa de identidade visual forte — o “cheiro de Bootstrap” é real e reconhecível
- Você só precisa de três componentes: o peso não compensa
- O time já mantém um design system próprio

A ALTERNATIVA DE MERCADO

Tailwind CSS

Utility-first: você não escreve CSS, você compõe classes atômicas no HTML.

Hoje é a escolha dominante em produto novo. Vale muito a pena aprender — depois desta semana.

POR QUE NÃO AGORA

Dois motivos honestos

- Exige etapa de build, e isso rouba tempo do objetivo real da semana
- Ele dissolve o conceito de “componente CSS” — que é exatamente o que queremos que você aprenda a construir

Toda escolha de ferramenta tem um custo. Saber dizer qual é o custo da sua escolha é o que separa o profissional do repetidor de tutorial.

O cartão responsivo

Abra o DevTools no modo dispositivo (Ctrl+Shift+M) e trabalhe em 375px de largura o tempo todo.

1 TODO CSS-1

A lista vira 2 colunas em 768px e 3 em 1200px. Só ``min-width``.

2 TODO CSS-2

Crie ``patient-card--inactive`` usando os tokens — nada de hexadecimal solto.

3 Não dependa só da cor

Combine cor + forma + texto. Cerca de 8% dos homens têm alguma deficiência na percepção de cores.

4 Teste nos três tamanhos

375px, 768px e 1280px. Nada de rolagem horizontal em 375px.

CRITÉRIO DE PRONTO

Checklist visual

- 1 / 2 / 3 colunas conforme a largura
- Cartão inativo distinguível sem depender da cor
- Nenhum valor de cor ou espaço fora dos tokens
- Zero rolagem lateral no celular

NÍVEL 3

Para quem terminou

- Faça o cartão inteiro receber foco por teclado (``:focus-visible``)
- Adicione um modo escuro trocando só os tokens em ``:root``

FERRAMENTAS PROFISSIONAIS

Programar com IA

Permitido, incentivado — e com protocolo. Como qualquer ferramenta séria.



O que pode, o que pode com cuidado, e o que não pode

ZONA VERDE · USO LIVRE

IA como professor particular

- “Explique a diferença entre ``map`` e ``forEach``”
- “O que faz esta linha do código-base?”
- “Traduza esta mensagem de erro”
- “Sugira nomes melhores para esta função”

ZONA AMARELA · COM VALIDAÇÃO

IA como colega que erra

- Gerar rascunho de função de render
- Gerar CSS de um componente
- Gerar a estrutura de uma rota

Zona amarela exige: protocolo V.E.R.M. + registro no IA.md

ZONA VERMELHA · NÃO PERMITIDO

IA como substituta

- Colar o enunciado e entregar a saída
- Entregar código que você não sabe explicar
- Aceitar biblioteca fora da lista da semana

Nenhuma dessas regras é sobre desconfiar de você. São sobre o que você vai saber fazer quando não houver assistente disponível.

C · P · T · R · F · A

PROMPT BEM CONSTRUÍDO

[CONTEXTO] Projeto em JavaScript puro, sem framework.
Ja tenho state.js com o estado e render.js que desenha.

[PAPEL] Aja como revisor de codigo frontend senior.

[TAREFA] Escreva a funcao getVisiblePatients().

[RESTRICOES] Sem bibliotecas. Sem framework. Sem acessar o DOM. Funcao pura. Nao altere o objeto state.

[FORMATO] So a funcao, com comentarios em portugues.

[ACEITACAO] Deve combinar busca por nome e filtro de ativos, e devolver array vazio quando nada bate.

O PROMPT RUIM

“faz um crud de paciente em js”

Devolve React, Prisma, camadas e autenticação. Tudo fora do escopo.

O ELEMENTO DECISIVO

RESTRIÇÕES

É a parte que quase ninguém escreve — e a que mais muda o resultado.

A IA não sabe o que você já viu em aula. Sem restrição explícita, ela responde para o programador médio da internet, não para você.

Antes de qualquer código gerado entrar no projeto

V

Verifica

Roda? Faz exatamente o que eu pedi?

Se falhar: devolva o erro REAL para a IA, não "não funcionou".

E

Explica

Consigo explicar cada linha para minha dupla?

Se falhar: peça explicação linha a linha ANTES de usar.

R

Reduz

Tem algo aqui que eu não preciso?

Import inútil, try/catch vazio, biblioteca extra: apague.

M

Marca

Registrei no IA.md?

Ferramenta, objetivo, o que veio, o que eu decidi, como validei.

Regra de bolso: se a IA te deu mais arquivos do que você tinha, ela respondeu a outra pergunta.

O que a IA vai errar nesta stack — e isso não é defeito dela

Ela responde com o que é mais comum na internet. O mais comum não é o nosso escopo.

- ! Vai sugerir React para qualquer coisa de frontend
- ! Vai sugerir jQuery se você mencionar “manipular o DOM”
- ! Vai propor uma arquitetura em camadas que ainda não temos
- ! Vai escrever ``max-width`` em vez de ``min-width``
- ! Vai importar uma biblioteca de datas para formatar uma data
- ! Vai devolver um ``try/catch`` que engole o erro em silêncio

Saber onde a ferramenta erra é o que transforma você em quem MANDA nela. Quem não conhece os modos de falha só consegue obedecer ao que ela sugere.

Três linhas, no papel ou no formulário

1

Em uma frase: por que separar estado de renderização?

Escreva com suas palavras. Se a frase sair igual à do slide, você decorou — tente de novo.

2

O que ficou nebuloso hoje?

Isso define os primeiros 6 minutos do próximo encontro. Ser honesto aqui muda a próxima aula.

3

Uma coisa que a IA te disse hoje e você recusou.

Se você não recusou nada, ou teve muita sorte, ou não estava validando.

Sem nota. É instrumento de diagnóstico, não de avaliação.

Duas coisas — e uma pergunta para levar na cabeça

OBRIGATÓRIO · 20 MIN

Termine as três práticas

- Se ficou lacuna, resolva em casa
- A solução do Encontro 1 será publicada no branch `aula1-solucao` HOJE à noite
- Compare com a sua — e entenda as diferenças, não copie

LEITURA · 10 MIN

MDN: HTTP Messages

- Só as seções de request e response
- Não precisa decorar nada
- Chegue sabendo o nome das partes: método, caminho, cabeçalho, corpo, status

A pergunta para levar: aquele arquivo `mock/patients.json` — de onde ele deveria vir de verdade?
Quem escreve? Quem guarda? O que acontece quando duas pessoas cadastram ao mesmo tempo?
É isso que resolvemos no próximo encontro.

O QUE LEVAR PARA CASA

Quatro frases

- 1 O estado muda. A tela é consequência.
- 2 Se dá para calcular, não guarde: derive.
- 3 Todo ``fetch`` mora em um arquivo só.
- 4 Mobile-first: ``min-width`` acrescenta, ``max-width`` desfaz.

Próximo encontro: HTTP, JSON e um banco de dados de verdade.